

第七次上机作业

学号：241830137 姓名：朱子健

2026 年 4 月 22 日

1 问题

实现快速傅里叶变换 FFT，并应用于某个具体场景。如信号，图像，声音处理，多项式乘法，偏微分方程求解 (谱方法)，或其它应用场景。

2 算法原理与设计

2.1 算法原理

2.1.1 快速傅里叶变换

离散 Fourier 变换（或逆变换）可以归结为计算

$$c_k = \sum_{j=0}^{N-1} x_j \omega^{kj} (k = 0, 1, \dots, N-1)$$

其中

$$\omega = e^{-i\frac{2\pi}{N}}$$

或

$$\omega = e^{i\frac{2\pi}{N}}$$

$x_j (j = 0, 1, \dots, N-1)$ 为已知复数序列。计算全部 c_k 共需 N^2 个操作。当 N 很大时计算量也很大，直到上个世纪 60 年代中期 Cooley & Tukey 提出了快速傅里叶变换 (FFT) 算法，可将计算工作量降到大约 $O(N \log N)$ ，才大大提高了运算速度。FFT 算法的基本思想是尽量减少乘法的次数，例如 $ab + ac + ad = a(b + c + d)$ ，左端三次乘法而右端只用一次乘法。实际上 $e^{i\frac{2\pi}{N}j} (k, j = 0, 1, \dots, N-1)$ 只有 N 个不同的值。特别是 $N = 2^P$ 时只有 $\frac{N}{2}$ 个不同的值。因此在计算 c_k 时，可合并大量同因子的项，从而减少乘法次数。

2.1.2 用快速傅里叶变换实现多项式乘法

设两个多项式

$$A(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}, \quad B(x) = b_0 + b_1x + \dots + b_{m-1}x^{m-1},$$

其乘积 $C(x) = A(x) \cdot B(x)$ 的系数为卷积：

$$c_k = \sum_{i=0}^k a_i b_{k-i}, \quad k = 0, 1, \dots, n+m-2.$$

直接计算的时间复杂度为 $O(nm)$ 。利用快速傅里叶变换 (FFT) 可降至 $O(N \log N)$ ，其中 N 为不小于 $n + m - 1$ 的 2 的幂。核心依据是卷积定理：

$$\mathcal{F}\{A * B\} = \mathcal{F}\{A\} \cdot \mathcal{F}\{B\},$$

即在点值表示下，多项式乘积对应点值逐点相乘。

2.1.3 算法设计

算法 1 快速傅里叶变换实现方法

- 1: Step 1 给出复数数组 $A_0(N), A_1(N)$ 及 $w(\frac{N}{2})$ ，确定 $P, q = 1$ 。将已知数据 $\{x_j\}$ 输入到数组 $A_0(N)$ 的单元中。
- 2: Step 2 计算 $w^m = e^{-i\frac{2\pi}{N}m}$ (或 $w^m = e^{i\frac{2\pi}{N}m}$) (m 从 0 到 $2^{P-1} - 1$)，结果存放在 $w(\frac{N}{2})$ 的相应单元中。
- 3: Step 3 j 从 0 到 $(2^{P-q} - 1)$ ， k 从 0 到 $(2^{q-1} - 1)$ ，按 FFT 公式 (3.5.15) 计算 A_q ，结果放在 $A_1(N)$ 的相应单元中。

4:

$$\begin{cases} A_1(j2^q + k) = A_0(j2^{q-1} + k) + A_0(j2^{q-1} + k + 2^{P-1}) \\ A_1(j2^q + k + 2^{q-1}) = [A_0(j2^{q-1} + k) - A_0(j2^{q-1} + k + 2^{P-1})]w(j2^{q-1}) \end{cases}$$

5: Step 4 k, j 循环结束， $q + 1 \rightarrow q, A_1(N) \rightarrow A_0(N)$ 。

6: 若 $q < P$ ，则 goto Step 3；否则 goto Step 5。

7: Step 5 此时 $q = P, j = 0, k$ 从 0 到 $(2^{P-1} - 1)$ 。计算

8:

$$\begin{cases} A_1(k) = A_0(k) + A_0(k + 2^{P-1}) \\ A_1(k + 2^{q-1}) = A_0(k) - A_0(k + 2^{P-1}) \end{cases}$$

9: 这里得到的 $A_1(k) (k = 0, 1, \dots, 2^{P-1})$ 即为所求 c_k 。

算法 2 快速傅里叶变换实现多项式乘法

输入: 多项式 $A(x) = \sum_{i=0}^{n-1} a_i x^i$, $B(x) = \sum_{j=0}^{m-1} b_j x^j$, 系数为整数或实数

输出: 乘积多项式 $C(x) = A(x)B(x) = \sum_{k=0}^{n+m-2} c_k x^k$ 的系数数组 $c[0..n+m-2]$

```
1: 令  $L = n + m - 1$ ,  $N = 2^{\lceil \log_2 L \rceil}$ 
2: 构造数组  $A_{\text{pad}}[0..N-1]$ ,  $B_{\text{pad}}[0..N-1]$ , 初值为 0
3: for  $i = 0$  至  $n - 1$  do
4:    $A_{\text{pad}}[i] \leftarrow a_i$ 
5: end for
6: for  $j = 0$  至  $m - 1$  do
7:    $B_{\text{pad}}[j] \leftarrow b_j$ 
8: end for
9:  $\hat{A} \leftarrow \text{FFT}(A_{\text{pad}})$ 
10:  $\hat{B} \leftarrow \text{FFT}(B_{\text{pad}})$ 
11: for  $k = 0$  至  $N - 1$  do
12:    $\hat{C}[k] \leftarrow \hat{A}[k] \cdot \hat{B}[k]$ 
13: end for
14:  $C_{\text{pad}} \leftarrow \text{IFFT}(\hat{C})$ 
15: for  $k = 0$  至  $L - 1$  do
16:    $c_k \leftarrow \text{round}(\text{Re}(C_{\text{pad}}[k]))$ 
17: end for
18:
19: return  $c[0..L-1]$ 
```

3 结果分析

表 1: FFT 多项式乘法示例

编号	多项式 A(x)	多项式 B(x)	乘积 C(x)
1	$1 + 2x + 3x^2$	$4 + 5x + 6x^2$	$4 + 13x + 28x^2 + 27x^3 + 18x^4$
2	$1 - x + x^2 - x^3$	$1 + x + x^2 + x^3$	$1 - x^6$
3	$\sum_{i=0}^9 (i+1)x^i$	$\sum_{i=0}^9 (i+1)x^i$	$1 + 4x + 10x^2 + 20x^3 + 35x^4 + 56x^5 + 84x^6 + 120x^7 + 165x^8 + 220x^9 + 276x^{10} + 324x^{11} + 360x^{12} + 380x^{13} + 380x^{14} + 360x^{15} + 324x^{16} + 276x^{17} + 220x^{18} + 165x^{19} + 120x^{20} + 84x^{21} + 56x^{22} + 35x^{23} + 20x^{24} + 10x^{25} + 4x^{26} + x^{27}$
4	$2 + 3x + 5x^2 + 7x^3$	$11 + 13x + 17x^2 + 19x^3$	$22 + 59x + 128x^2 + 254x^3 + 317x^4 + 337x^5 + 254x^6 + 1$

4 附录: Python 源代码实现

```
1 import math, cmath
2 def fft(a, invert):
3     n = len(a)
```

```

4     if n == 1:
5         return a
6     # 分治: 奇偶下标
7     even = fft(a[0::2], invert)
8     odd  = fft(a[1::2], invert)
9     # 合并
10    ang = 2 * math.pi / n * (-1 if invert else 1)
11    w = 1
12    wn = complex(math.cos(ang), math.sin(ang))
13    for i in range(n // 2):
14        t = w * odd[i]
15        a[i] = even[i] + t
16        a[i + n//2] = even[i] - t
17        w *= wn
18    if invert:
19        for i in range(n):
20            a[i] /= n
21    return a
22
23 def poly_mul_fft(a, b):
24     n = 1
25     while n < len(a) + len(b) - 1:
26         n <= 1
27         fa = a + [0] * (n - len(a))
28         fb = b + [0] * (n - len(b))
29         fft(fa, False)
30         fft(fb, False)
31         for i in range(n):
32             fa[i] *= fb[i]
33         fft(fa, True)
34     return [int(round(fa[i].real)) for i in range(len(a) + len(b) - 1)]

```

```

1 import numpy as np
2 def poly_mul_fft(a, b):
3     total_len = len(a) + len(b) - 1
4     n = 1
5     while n < total_len:
6         n <= 1
7         fa = np.fft.fft(a + [0] * (n - len(a)))
8         fb = np.fft.fft(b + [0] * (n - len(b)))
9         fc = fa * fb
10        c = np.fft.ifft(fc).real
11        return [int(round(x)) for x in c[:total_len]]
12 def main():
13     print("多项式乘法 (使用 FFT)")
14     print("请输入第一个多项式的系数列表, 例如 [1,2,3]: ")
15     a = eval(input().strip())
16     print("请输入第二个多项式的系数列表: ")
17     b = eval(input().strip())
18     c = poly_mul_fft(a, b)

```

```
19     print("乘积系数: ", c)
20 if __name__ == "__main__":
21     main()
```